Ecommerce Database Schema

Spring Boot backend + React frontend • Core tables, key fields, and relationships

1. Core Tables

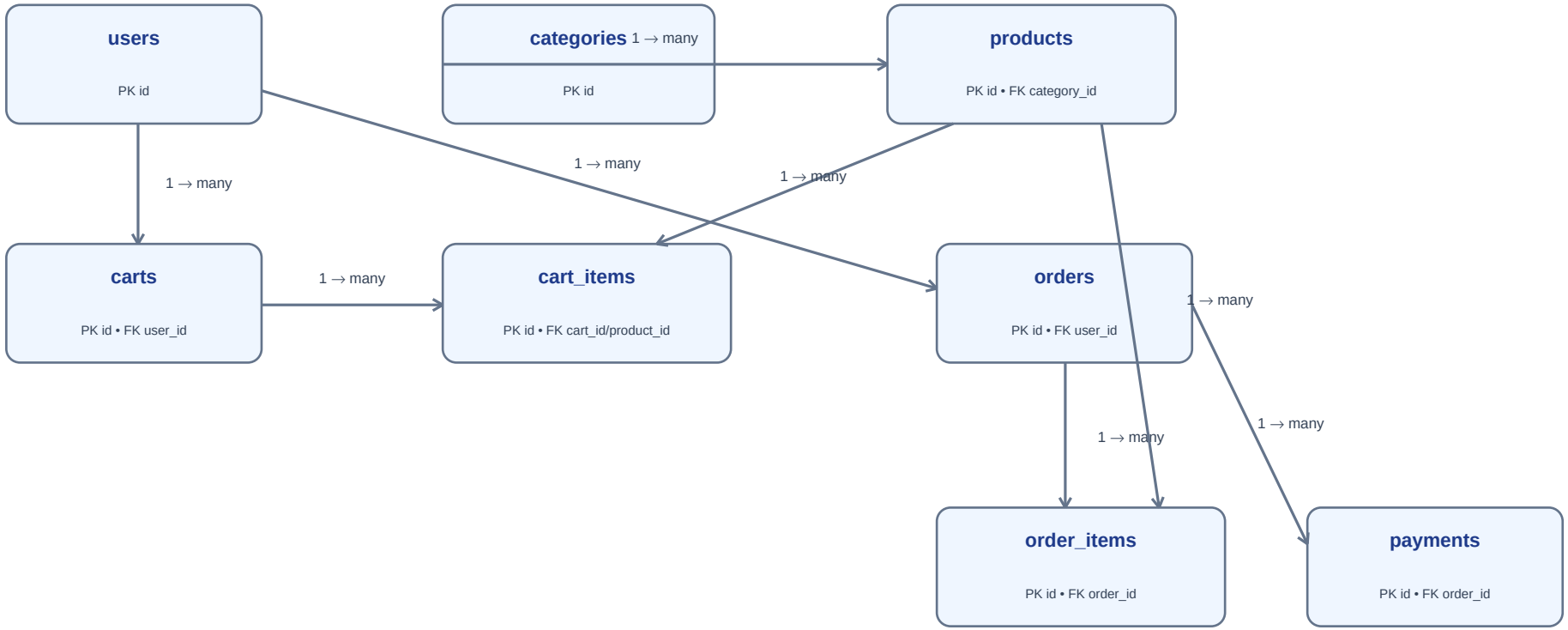
This is a practical starting schema for a portfolio ecommerce application. **PK** = primary key, **FK** = foreign key. Fields shown are suggested, not exhaustive.

Table	Purpose	Suggested fields
users	Customer accounts and roles	**PK** id; name; email (unique); password_hash; role; created_at
categories	Groups products into categories	**PK** id; name; description
products	Products offered for sale	**PK** id; name; description; price_cents; stock_quantity; **FK** category_id; image_url; created_at
carts	A customer's active shopping cart	**PK** id; **FK** user_id; created_at; updated_at
cart_items	Products and quantities in a cart	**PK** id; **FK** cart_id; **FK** product_id; quantity
orders	Submitted purchase and order totals	**PK** id; **FK** user_id; status; subtotal_cents; tax_cents; shipping_cents; total_cents; created_at
order_items	Snapshot of items purchased	**PK** id; **FK** order_id; product_id; product_name; unit_price_cents; quantity; line_total_cents
payments	Payment attempts and outcomes	**PK** id; **FK** order_id; amount_cents; status; provider; provider_transaction_id; created_at

Design notes: Keep prices as integer cents (e.g., 2599 = $25.99) or use SQL DECIMAL with Java BigDecimal; never use float/double for money. Save the price and product name on order_items so past orders remain accurate if a product changes later. Never store raw card numbers or CVVs.

2. Table Relationships

Crow's-foot notation: one-to-many relationships are shown as 1 → many. The foreign key lives on the “many” side.



Note: product_id on order_items may be nullable for deleted products, while product_name and unit_price_cents preserve the purchase snapshot.

Relationship	Meaning
users → carts	A user can have one or more carts over time; enforce one active cart per user if desired.
users → orders	A user can place many orders.
categories → products	A category can contain many products; each product references one category in this design.

Relationship	Meaning
carts → cart_items	A cart contains multiple line items.
products → cart_items	A product can appear in many carts.
orders → order_items	An order contains one or more purchased line items.
products → order_items	An item references a product but also stores a snapshot of its name and purchase-time price.
orders → payments	An order can have multiple payment attempts, such as a failed attempt followed by a successful one.

3. Suggested Build Order

Step	Build	Why
1	categories, products	You need products before you can display a catalog.
2	users	Add authentication and ownership for customer data.
3	carts, cart_items	Let customers add, remove, and update product quantities.
4	orders, order_items	Create a durable purchase record from the cart at checkout.
5	payments	Integrate payment-provider test mode and record verified payment outcomes.

Optional tables to add later

Table	Use it when...
addresses	Customers need saved shipping/billing addresses. For a simpler first version, snapshot the shipping address on orders.
tax_rates	Your application needs to manage tax jurisdictions and effective dates. Always save the calculated tax amount on the order.
shipments	You want shipment tracking, carrier details, and delivery statuses.
reviews	Customers can review products they purchased.
wishlists / wishlist_items	Customers can save products for later.
refunds	You need detailed refund records separate from the original payment, depending on your payment-provider model.

Checkout rule: React may display totals, but Spring Boot must recalculate prices, tax, shipping, and the final amount. Verify payment status on the backend (for example, through a verified payment-provider webhook) before marking an order as paid.